

The STL Vector Class

CSC 240 Data Structures Fall 2025

Marcus Birkenkrahe

November 28, 2025

Contents

1	Objectives	2
2	Overview	2
3	Vector constructors	3
4	List of vector member functions	4
5	Minimal vector API for Data Structures	7
6	PRACTICE Review of basic vector operations	7
7	PRACTICE Find the error and correct the code	9
8	Using an iterator with vector	10
9	PRACTICE Simplify an iterator	11
10	Inserting new elements into a vector	12
11	PRACTICE Insert a single value in a vector	14
12	PRACTICE Insert a range of elements into a vector	16
13	Storing class objects in a vector	18
14	Displaying vectors with class objects using an iterator	20
15	PRACTICE Adding class objects to an existing vector	21

16 Inserting elements without first creating objects	24
17 Storage allocation functions for vector	27
18 PRACTICE Demonstrate vector storage allocation functions	28
19 PROJECT Modifying containers while iterating [CSC 450]	32
20 Bonus: What does "theoretical storage limit" mean?	37
21 PRACTICE Review questions	38
22 Glossary	41

1 Objectives

The `vector` is an example of a **linked list** data structure.

- ☐ Overview of `vector`
- ☐ `vector` constructors
- ☐ `vector` member functions
- ☐ Minimal `vector` API for Data Structures
- ☐ Review of basic `vector` operations
- ☐ Using an iterator with `vector`
- ☐ Inserting new elements into a `vector`
- ☐ Storing `class` objects in a `vector`
- ☐ Storage allocation functions

2 Overview

- What is a `vector`?

A `vector` is a sequential container of elements of the same data type: Integers, doubles, characters, strings, classes, etc.

- Why use `vector` instead of arrays?

1. You don't have to declare the size of a `vector` when you define it.
 2. You can add elements to, or delete from it at runtime, and the `vector` automatically adjusts its size.
 3. A `vector` can report the number of elements it contains.
- What do you have to do to use `vector`?

To use `vector` you have to `#include <vector>` and if you want to avoid having to write `std::vector`, you need to add:
`using namespace std;`

3 Vector constructors

- **Tip:** Check `std::vector` at cppreference.com before and after this section to see if your understanding changed!
- `vector` has five constructors, that is five different ways of creating containers. We illustrate these with working code.

1. **Default constructor:** creates an empty `vector`

```
vector<string> names; // an empty vector of string objects
names.push_back("Jim");
names.push_back("Jane");
names.push_back("Joe");
for (auto name : names)
    cout << name;    // name is NOT an iterator - it is a copy of each vector el
// behind the loop an iterator std::vector<std::string>::iterator is used!
// names.begin() to names.end()

JimJaneJoe
```

2. **Fill constructor:** creates a `vector` of length `size`

```
vector<bool> values (3, 0); // a vector of three booleans (false)
for (auto elem : values) cout << boolalpha << elem << " ";

false false false
```

3. **Range constructor:** creates a `vector` of a range of values specified by two iterators.

```

vector<int> vec {1,2,3,4,5,6,7,8};
auto it1 = vec.begin() + 2; // 3
auto it2 = vec.end() - 2;   // 7
cout << *it1 << " " << *it2 << endl;

vector<int> vec2(it1,it2); // sub-range from it1 (3) to it2 (7)
for (auto elem : vec2) cout << elem << " ";

3 7
3 4 5 6

```

4. **Copy constructor:** creates a **vector** by copying another **vector**

```

vector<string> names = {"Jimmy", "Johnny", "Jenny"};
vector<string> names2(names); // copy of names vector
for (auto elem : names2) cout << elem << " ";

```

Jimmy Johnny Jenny

4 List of vector member functions

There are 40 (!) member functions in C++20 (including the constructors and overloaded operators). We order member functions by category:

1. Construction and assignment
2. Element access
3. Iterator support
4. Capacity operations
5. Modifiers
6. Allocator
7. Comparisons

This **function overload** is intentional: STL containers are designed for performance, generality, and type safety, favoring completeness over simplicity. C takes the opposite view: Simple over safe and complete.

1. **Construction & Assignment**

Function	Description
constructor	Creates vector (default, copy, move, range, init)
operator=	Assigns contents
assign(size, value)	Replaces contents from range or count+value
assign(iterator1, iterator2)	

2. Element Access

Function	Description
at(index)	Returns reference to element with bounds checking
operator[]	Returns reference to element without bounds checking
front()	Returns reference to first element
back()	Returns reference to last element
data()	Returns pointer to first element in container

3. Iterators¹

Function	Description
begin()	Iterator to first element
end()	Iterator past last element
cbegin()	Const iterator to first element
cend()	Const iterator past last element
rbegin()	Reverse iterator to last element
rend()	Reverse iterator before first
crbegin()	Const reverse iterator to last
crend()	Const reverse iterator before first

4. Capacity

¹`const iterator` allows read-only access to container elements. It cannot be used to modify elements it points to.

Function	Description
<code>empty()</code>	Returns true if container is empty
<code>size()</code>	Returns number of elements
<code>max_size()</code>	Max number of elements (theoretically) possible
<code>capacity()</code>	Returns number of elements that underlying array can hold, without reallocating the array ²
<code>resize(n)</code>	Changes number of elements to n
<code>resize(n, value)</code>	Changes number of elements to n - if <code>size() < n</code> then the new elements are initialized with value
<code>reserve(n)</code>	Increases capacity to n (preallocation)
<code>shrink_to_fit()</code>	Requests to reduce <code>capacity()</code> to be equal <code>size()</code>

5. Modifiers

Function	Description
<code>clear()</code>	Erases all elements
<code>insert(it, n, value)</code>	Inserts n new elements at position it with that (single) value
<code>emplace(it, value)</code>	Constructs element in-place at position it (new element inserted before position it)
<code>erase(it)</code>	Removes element at position it and returns iterator pointing to the next element
<code>push_back(value)</code>	Adds element to end
<code>emplace_back(value)</code>	Constructs element at end
<code>pop_back()</code>	Removes last element
<code>swap(second)</code>	Swaps content of calling vector and of the second object (same data type)

6. Allocator

Function	Description
<code>get_allocator()</code>	Returns allocator (custom memory management)

7. Comparisons (C++20)

Function	Description
<code>operator==</code>	Equality comparison
<code>operator<=></code>	Three-way comparison (spaceship operator) ³

²For example, after `vec.reserve(100)`, the `vector` will not reallocate until more than 100 elements are added.

5 Minimal vector API for Data Structures

Fortunately, you don't need to memorize 40 functions - only these 10 are really critical for competently using `vector` containers:

	Category	Function	Purpose
1	Access	<code>operator[]</code>	Random access
2	Capacity	<code>size()</code>	Track current size
3	Capacity	<code>reserve()</code>	Preallocate storage
4	Capacity	<code>capacity()</code>	Know current storage
5	Modification	<code>push_back()</code>	Add element to end
6	Modification	<code>pop_back()</code>	Remove last element
7	Modification	<code>insert()</code>	Insert at position
8	Modification	<code>erase()</code>	Remove at position
9	Modification	<code>clear()</code>	Remove all elements
10	Iteration	<code>begin()</code> , <code>end()</code>	Traverse elements

6 PRACTICE Review of basic vector operations

- Read and understand the following program, run it, and then comment each section.

```
#include <iostream>
#include <vector>    // ...
using namespace std; // ...

int main()
{
    // ...
    const int SIZE = 10;

    // ...
    vector<int> numbers(SIZE);

    // ...
    for (int index = 0; index < numbers.size(); index++)
        numbers[index] = index;
```

³Instead of writing all six relational operators `<`, `<=`, `>`, `>=`, `==`, `!=`, you write `<=>` and the compiler generates the rest.

```

    // ...
    for (auto element : numbers)
        cout << element << " ";
    cout << endl;

    return 0;
}

```

0 1 2 3 4 5 6 7 8 9

0 1 2 3 4 5 6 7 8 9

- Solution:

```

#include <iostream>
#include <vector>    // Include vector class header file
using namespace std; // Use runtime STL std::

int main()
{
    // Define an integer constant.
    const int SIZE = 10;

    // Define a vector to hold 10 integer values.
    vector<int> numbers(SIZE);

    // Store the values 0 through 9 in the vector.
    for (int index = 0; index < numbers.size(); index++)
        numbers[index] = index;

    // Display the vector elements using a range-based loop.
    for (auto element : numbers)
        cout << element << " ";
    cout << endl;

    return 0;
}

```

0 1 2 3 4 5 6 7 8 9

7 PRACTICE Find the error and correct the code

1. What's the error? Can you fix it?

```
vector<int> numbers;  
numbers[0]=99;
```

Solution:

```
vector<int> numbers = {0};  
numbers[0]=99; // vector must be non-empty
```

2. What's the error? Can you fix it?

```
vector<int> numbers;  
push_back(10);
```

Solution:

```
vector<int> numbers;  
numbers.push_back(10); // push_back declared in numbers scope
```

3. What's the error? Can you fix it?

```
vector<bool> numbers;  
numbers.push_back(true);  
numbers.pop_back(false);
```

Solution:

```
vector<bool> truth;  
truth.push_back(true); // add 'true' at end of truth (truth[0])  
truth.pop_back();      // pop_back() has no arguments
```

4. What's the error? Can you fix it?

```
vector<string> names = {"Jimmy Dean", "Johnny Cash", "Jack Black"};  
cout << names.at(3) << endl;
```

Solution:

```
vector<string> names = {"Jimmy Dean", "Johnny Cash", "Jack Black"};
cout << names.at(2) << endl; // at(3) throws an out_of_range exception
```

Jack Black

5. What's the error? Can you fix it?

```
vector<string> names = {"Jimmy Dean", "Johnny Cash", "Jack Black"};
vector<string> names2;
names2[0] = names[0];
names2[1] = names[1];
names2[2] = names[2];
```

Solution:

```
vector<string> names = {"Jimmy Dean", "Johnny Cash", "Jack Black"};
vector<string> names2;
// names2[0] = names[0]; // cannot assign to names2 before it exists
names2 = names; // overloaded assignment operator= for vector
for (auto element : names2)
    cout << element << " ";
```

Jimmy Dean Johnny Cash Jack Black

8 Using an iterator with vector

- The `vector` class supports the use of these iterators:
 1. `iterator` (forward iterator)
 2. `const_iterator` (constant iterator)
 3. `reverse_iterator` (backward iterator)
 4. `const_reverse_iterator` (constant backward iterator)
- Compare the member support functions for getting iterators: `begin()`, `end()`, `cbegin()`, `cend()`, `rbegin()`, `rend()`, `crbegin()`, `crend()`.
- You can simplify the definition of an iterator with `auto`. In the following code block loop 1 and loop 2 are equivalent:

```
vector<int> num {1,1,1,1,1};

// loop 1
vector<int>::iterator it;
for (it = num.begin(); it != num.end(); it++) {cout<<*it;}

cout << endl;
// loop 2
for (auto it = num.begin(); it != num.end(); it++) {cout<<*it;}

11111
11111
```

- Where does `it` point? How many times does the loop run?

Since `num` is empty, `num.begin()` equals `num.end()` and the loop never runs. `it` does not point to any valid element. No memory is reserved for `num` until elements are inserted or `reserve(n)` is called.

9 PRACTICE Simplify an iterator

- Create a `vector` that holds these `names`: Joe, Jim, Jack, Jane.
- Define a forward iterator `it` using the `iterator` keyword.
- Use `it` to display each element in the vector.
- When the program works, use another iterator `it2` using the `auto` keyword to display the elements again without explicitly defining the iterator, but in reverse order, using `rbegin()` and `rend()`!
- Hint: `rbegin()` begins at the reverse beginning (the end) of the `vector`, and `rend()` ends at its reverse end (the beginning).
- Sample output:

```
#+RESULTS:
: Joe Jim Jack Jane
: Jane Jack Jim Joe
```

- Solution:

```
vector<string> names = {"Joe", "Jim", "Jack", "Jane"};

vector<string>::iterator it;

for(it = names.begin(); it != names.end(); it++)
    cout << *it << " ";

cout << endl;

for(auto it2 = names.rbegin(); it2 != names.rend(); it2++)
    cout << *it2 << " ";

Joe Jim Jack Jane
Jane Jack Jim Joe
```

10 Inserting new elements into a vector

- To insert one or more new elements at a specified position in a **vector**, use **insert**. The full definition is complicated (see cppreference.com)!
- There are multiple overloads (alternative uses) but these three are most common:
 1. Insert a single **value** of type **T** at the specified position:


```
iterator insert(iterator pos, const T& value);
```
 2. Insert **count** copies of a **value**:


```
iterator insert(iterator pos, size_type count, const T& value);
```
 3. Insert a range of values:


```
iterator insert(iterator pos, iterator pos1, iterator pos2);
```
- The **iterator** is like a bookmark inside the vector. Insertion happens before the iterator position.
- Example: Inserting a single value in a numeric **vector**

```
vector<int> v{1,2,3,4,5};
for (auto element : v) cout << element << " "; cout << endl; // print vector
auto it = v.begin() + 2; // points to element 4
v.insert(it, 3);          // insert 3 before 4
for (auto element : v) cout << element << " ";
```

```
1 2 3 4 5
1 2 3 3 4 5
```

- The function returns an iterator to the newly inserted element:

```
vector<int> v {1,2,3,4,5};
auto inserted = v.insert(v.begin(), 99);
cout << *inserted << endl;
```

```
99
```

- Example: Insert multiple copies

```
vector<int> v {1,2,5};
v.insert(v.begin() + 2, 2, 3); // insert two 3s before 5
for (auto element : v) cout << element << " ";
```

```
1 2 3 3 5
```

- Example: Insert a range of values:

```
vector<int> a{10,20};
vector<int> b{1,2,3};
a.insert(a.begin() + 1, b.begin(), b.end()); // insert b before 20
for (auto element : a) cout << element << " ";
```

```
10 1 2 3 20
```

- Does `insert` modify a vector in-place or does it create a copy?

- The vector is modified in-place (you're not getting a new copy) but memory management does take place:
 - If the vector does not have enough **capacity**, a new, larger array is allocated, all elements are moved to the new memory, the old array is deleted, and all iterators, pointers, references become invalid.
- **Challenge:** What's the final vector in the following example? Guess the result before trying this yourself!

```
vector<char> letters {'A', 'C', 'D'};
letters.insert(letters.begin() + 1, 'B');
letters.insert(letters.end(), {'E', 'F', 'G'});

for (char letter : letters) cout << letter << " ";

A B C D E F G
```

11 PRACTICE Insert a single value in a vector

- Use the starter code below to
 1. Define an integer vector **numbers** with the numbers 1, 2, 3, 4, 5.
 2. Define an iterator **it** pointing to the second element of **numbers**.
 3. Insert a new element with the value 99.
 4. Display the vector elements.
- Sample output:

```
1 99 2 3 4 5
```

- Starter code: <https://tinyurl.com/insert-value>

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
```

```

// Define a vector 'numbers' with 5 integer values 1,2,3,4,5.

// Define an iterator 'it' pointing to the second element.

// Insert a new element with the value 99

// Display the vector elements.

return 0;
}

```

- Starter code: <https://tinyurl.com/insert-range>

```

#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // Define a vector 'numbers' with 5 integer values 1,2,3,4,5.
    vector<int> numbers = {1,2,3,4,5};
    // Define an iterator 'it' pointing to the second element.
    auto it = numbers.begin() + 1;
    // Insert a new element with the value 99
    numbers.insert(it,99);
    // Display the vector elements.
    for (int num : numbers) cout << num << " ";
    return 0;
}

```

1 99 2 3 4 5

- Modify the program to insert 4 numbers 99 before the 3rd element of the original vector. Sample output: 1 2 99 99 99 99 3 4 5.
- Solution:

```

#include <iostream>
#include <vector>

```

```

using namespace std;

int main()
{
    // Define a vector 'numbers' with 5 integer values 1,2,3,4,5.
    vector<int> numbers = {1,2,3,4,5};
    // Define an iterator 'it' pointing to the second element.
    auto it = numbers.begin() + 2;
    // Insert a new element with the value 99
    numbers.insert(it,4,99);
    // Display the vector elements.
    for (int num : numbers) cout << num << " ";
    return 0;
}

1 2 99 99 99 99 3 4 5

```

12 PRACTICE Insert a range of elements into a vector

- Use the starter code below to
 1. Define **two** integer vectors `v1` and `v2` with the elements `{1,2,3}` and the elements `{100,200,300,400,500}`, respectively.
 2. Define **three** iterators `it1`, `it2`, and `it3`:
 - `it1` points at 2 in `v1`
 - `it2` points at 100 in `v2`
 - `it3` points at 400 in `v2`.
 3. Insert the range of elements between `it2` and `it3` into `v1`
 4. Display the elements of `v1`

- Sample output:

```
1 100 200 300 2 3
```

- Starter code: <https://tinyurl.com/insert-range>

```

#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // Define two vectors v1, v2.

    // Define three iterators it1, it2, it3.

    // Insert a range of elements into v1.

    // Display the elements of v1.

    return 0;
}

```

- Solution:

```

#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // Define two vectors v1, v2.
    vector<int> v1{1,2,3};
    vector<int> v2{100,200,300,400,500};

    // Define three iterators it1, it2, it3.
    auto it1 = v1.begin() + 1;
    auto it2 = v2.begin();
    auto it3 = v2.begin() + 3;

    // Insert a range of elements into v1.
    v1.insert(it1, it2, it3);

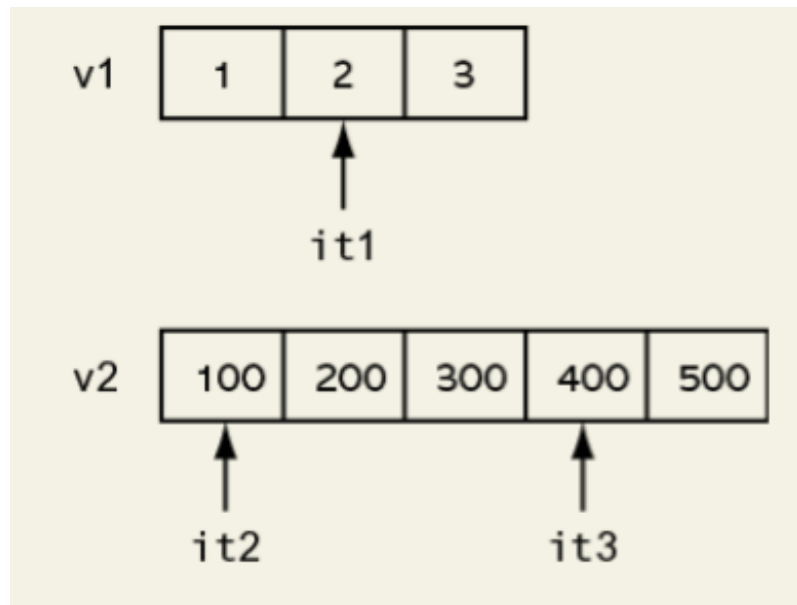
    // Display the elements of v1.
    for (auto num : v1) cout << num << " ";
}

```

```
    return 0;
}
```

```
1 100 200 300 2 3
```

- Visualization:



13 Storing class objects in a vector

- Remember that a `class` is a user-defined data structure. It can be used like a type when defining a `vector`.
- The file `Product.h` contains the definition of the `Product` class. Below, we will define and initialize a `vector` to hold `Product` objects.
- `Product.h`: Products have a `private name` and `units`. You can set both of these with `setName` and `setUnits`, and you can get their values with `getName` and `getUnits`.

```
#ifndef PRODUCT_H
#define PRODUCT_H
```

```

#include <string>
using namespace std;

class Product
{
private:
    string name;
    int units;
public:
    Product(string n, int u) { name = n; units = u;}
    void setName(string n) { name = n; }
    void setUnits(int u) { units = u; }
    string getName() const { return name; }
    int getUnits() const { return units; }
};
#endif

```

- Next, we `#include` the `Product.h` file in the main program, create a few `Product` objects, store them in a `vector`, and print their data. The file is called `Products.cpp`.

```

#include <iostream>
#include <vector>
#include "Product.h"
using namespace std;

int main()
{
    // Create a vector of Product objects.
    vector<Product> products =
    {
        Product("T-Shirt", 20),
        Product("Calendar", 25),
        Product("Coffee Mug", 30)
    };

    // Display the vector elements.
    for (auto product : products)
    {
        cout << "Product: " << product.getName() << endl

```

```

        << "Units:  " << product.getUnits() << endl;
    }
    return 0;
}

```

```

Product: T-Shirt
Units: 20
Product: Calendar
Units: 25
Product: Coffee Mug
Units: 30

```

14 Displaying vectors with class objects using an iterator

- Let's create a simple `Student` class, create a couple of students with a `name` and an `id`, add them to a `vector` of `Student` objects named `roster` and display the contents of that vector using an iterator on `getName()` and `getID()` functions.
- The main thing to remember is that the iterator works like a pointer to the container. So to use a member function like `getName()` on the container, we would call `roster.getName()` but to call it on the iterator `it` that points at `roster`, we would call: `it->getName()`.

```

#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Student {
private:
    string name;
    int id;
public:
    Student(const string& n, const int i) : name(n), id(i) {}
    string getName() { return name; }
    int getID() { return id; }
}

```

```

};

int main()
{
    // Create a vector of Student objects.
    vector<Student> roster = {
        Student("Alice", 1001),
        Student("Bob", 1002)
    };

    // Use an iterator to display the vector contents.
    for (auto it = roster.begin(); it != roster.end(); it++)
    {
        cout << "Student name = " << it->getName() << endl // (*it).getName
            << "Student ID = " << it->getID() << endl; // (*it).getID
    }
    return 0;
}

Student name = Alice
Student ID = 1001
Student name = Bob
Student ID = 1002

```

- Review: Why are we using `string& n` in the `Student` constructor instead of `string n`?

Passing `const std::string& n` avoids copying the `string` argument.

- Review: Why do we call `it->getName()` instead of `it.getName()`?

The iterator `it = roster.begin();` has no member `getName()` - it's a pointer that points at `roster` (which does have this member function). So both `it->getName()` or `(*it).getName()` are OK.

15 PRACTICE Adding class objects to an existing vector

- Modify the starter code below as follows:

1. Include the necessary header files
 2. Create three **Product** objects **prod1** (20 T-Shirts), **prod2** (25 calendars), **prod3** (30 coffee mugs).
 3. Create an empty vector **products** to hold the Products.
 4. Add the products to the vector using **push_back**.
 5. Use an iterator to display the vector contents.
- OneCompiler.com only allows single source files. Therefore, the definition of the **Product** class is included in **Main.cpp**.
 - Sample output:

```
Product: T-Shirt
Units:    20
Product: Calendar
Units:    25
Product: Coffee Mug
Units:    30
```

- Starter code: <https://tinyurl.com/vector-class-objects>

```
// Header files

using namespace std;

// Class definition
class Product
{
private:
    string name;
    int units;
public:
    Product(const string n, const int u) { name = n; units = u;}
    string getName() const { return name; }
    int getUnits() const { return units; }
};

int main()
{
    // Create Product objects.
```

```

    // Create an empty vector 'products' to hold the Products.

    // Add the products to the vector using 'push_back'

    // Use an iterator to display the vector contents.

    return 0;
}

```

- Solution:

```

// Header files
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// Class definition
class Product
{
private:
    string name;
    int units;
public:
    Product(const string& n, const int u) { name = n; units = u;}
    string getName() const { return name; }
    int getUnits() const { return units; }
};

int main()
{
    // Create Product objects.
    Product prod1("T-Shirt",20);
    Product prod2("Calendar",25);
    Product prod3("Coffee Mug",30);

    // Create an empty vector 'products' to hold the Products.
    vector<Product> products;
}

```

```

// Add the products to the vector.
products.push_back(prod1);
products.push_back(prod2);
products.push_back(prod3);

// Use an iterator to display the vector contents.
for (auto it=products.begin(); it != products.end(); it++)
{
    cout << "Product: " << it->getName() << endl
         << "Units:   " << it->getUnits() << endl;
}
return 0;
}

```

```

Product: T-Shirt
Units:   20
Product: Calendar
Units:   25
Product: Coffee Mug
Units:   30

```

16 Inserting elements without first creating objects

- For programs that make a lot of insertions, `insert` and `push_back` can be inefficient.

With C++, two new functions, `emplace` and `emplace_back`, you do not need to instantiate (create) the object that you're going to insert. Instead, pass any arguments normally passed to the constructor, and the functions handle the rest.

- Code example for `emplace_back` using the `Product` class: We're adding it as a header file - where does this file need to be to be found?

Answer: Not in the `$PATH` - that's for binaries/executables - the compiler first searches in the current directory. You can specify relative paths, as in `"../src/Product.h"` or absolute paths (discouraged - why?) like `"/home/user/src/Product.h"`. Or you can include the header files on the command-line with the `-I` flag:

```
g++ -I ../src main.cpp -o main # include files from '../src', output as 'main'
```

```
#include <iostream>
#include <vector>
#include "Product.h"
using namespace std;

int main()
{
    // Create a vector to hold Products.
    vector<Product> products;

    // This is where with 'push_back' you would have to create Products.
    // The emplace_back function does that for you now.

    // Add Products to the vector.
    products.emplace_back("T-Shirts",20);
    products.emplace_back("Calendar",25);
    products.emplace_back("Coffee Mug",30);

    // Display vector content.
    for (const auto& product : products)
    {
        cout << "Product: " << product.getName() << endl
              << "Units: " << product.getUnits() << endl;
    }
    return 0;
}
```

```
Product: T-Shirts
Units: 20
Product: Calendar
Units: 25
Product: Coffee Mug
Units: 30
```

- Is there any advantage from using the range-based `for` loop vs. the iterator-controlled `for` loop?

- Use the simple range-based loop when you want to read or modify each element directly and you don't need the index or change the container structure (e.g. erase elements).
 - Use the iterator-controlled loop when you need fine control over iteration (by index) and when you want to change container structure.
- Does it matter if you use `auto`, `auto&` or `const auto&` in a container loop?

Syntax	Meaning	When to Use
<code>auto</code>	Copy each element	Copying is cheap
<code>auto&</code>	Non-const ref to each element	Need to modify
<code>const auto&</code>	Const ref to each element	Don't need to modify

- The `emplace` member function constructs an object at a specified location in the `vector`. Its first argument is an iterator, and the new element will be constructed at the position right before the iterator. After the iterator, you pass the list of arguments to be forwarded to the new object's constructor.
- Program example:

```
#include <iostream>
#include <vector>
#include "Product.h"
using namespace std;

int main()
{
    // Create a vector to hold two Products.
    vector<Product> products =
    {
        Product("T-Shirt",20),
        Product("Coffee Mug",30)
    };

    // Get an iterator to the 2nd element
    auto it = products.begin() + 1;
```

```

// This is where with 'insert' you would have to create Products.
// The emplace_back function does that for you now.

// Insert another Product into the vector.
products.emplace(it, "Calendar", 25);

// Display vector content.
for (const auto& product : products)
{
    cout << "Product: " << product.getName() << endl
         << "Units: " << product.getUnits() << endl;
}
return 0;
}

```

```

Product: T-Shirt
Units: 20
Product: Calendar
Units: 25
Product: Coffee Mug
Units: 30

```

17 Storage allocation functions for vector

- The **vector** is a wrapper for a C-style array. When its array is full, and another element is added, the **vector** needs to
 1. allocate a new array with more memory
 2. copy all elements of the old array to the new array
 3. deallocate the old array.
- This wouldn't be very efficient if it had to be done every time a new element is added. Therefore, the **vector** object has more memory than it needs.
- A **vector** object has five important functions related to size:
 1. The number of elements that are stored in the **avector**: **size()**

2. The number of elements that can be stored in the `vector` without having to reallocate the underlying array, `capacity()`
3. The number of elements that the `vector` can ever store, `max_size()`
4. The `reserve` member function to increase the vector's capacity
5. The `shrink_to_fit()` function to decrease the vector's capacity so that it is the same as `size()`.

18 PRACTICE Demonstrate vector storage allocation functions

- Write a C++ program that demonstrates the storage allocation functions. Tip: Write a function to print vector size and capacity.
 1. Create an empty integer vector.
 2. Display the theoretical limit of the number of vector elements.
 3. Display size and capacity of an empty vector.
 4. Display size and capacity of a vector with 1,000 integers.
 5. Add another 1,000 integers to the vector and display size and capacity.
 6. Reserve space for 1,000 integers over capacity, and display size and capacity.
 7. Shrink the vector so that capacity equals size, and display size and capacity.
 8. Add one more element to the vector, and display size and capacity.
 9. Empty the vector and display size and capacity.
 10. What is the vector's capacity at the end?

- Solution:

```
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;

void prt(vector<int>&);
```

```

int main()
{
    // Create an empty integer vector nums.
    vector<int> nums;

    // Display the theoretical limit of the number of vector elements.
    cout << "Theoretical max number of vector elements = "
          << nums.max_size() << endl;

    // Display size and capacity of an empty vector.
    cout << "\nEmpty vector:\n";
    prt(nums);

    // Display size and capacity of a vector with 1,000 integers.
    cout << "\nVector for 1,000 integers:\n";
    for (int i=1;i<=1'000;i++)
        nums.push_back(i);
    prt(nums);

    // Add another 1,000 integers.
    cout << "\nVector for 2,000 integers:\n";
    for (int i=1;i<=1'000;i++)
        nums.push_back(i);
    prt(nums);

    // Reserve space for capacity() + 1,000 integers.
    cout << "\nVector for 3,000 integers (reserve):\n";
    nums.reserve(nums.capacity() + 1'000);
    prt(nums);

    // Shrink vector so that capacity equals size.
    cout << "\nShrink vector to fit only current size:\n";
    nums.shrink_to_fit();
    prt(nums);

    // Add one more element to the vector
    cout << "\nAdd one more element to the vector:\n";
    nums.push_back(1);
    prt(nums);
}

```

```

    // Empty the vector.
    cout << "\nEmpty the vector:\n";
    nums.clear();
    prt(nums);

    return 0;
}

void prt(vector<int>& v)
{
    cout << "Size = "      << v.size()      << endl
         << "Capacity = "  << v.capacity() << endl;
}

```

Theoretical max number of vector elements = 2305843009213693951

Empty vector:

Size = 0

Capacity = 0

Vector for 1,000 integers:

Size = 1000

Capacity = 1024

Vector for 2,000 integers:

Size = 2000

Capacity = 2048

Vector for 3,000 integers (reserve):

Size = 2000

Capacity = 3048

Shrink vector to fit only current size:

Size = 2000

Capacity = 2000

Add one more element to the vector:

Size = 2001

Capacity = 4000

Empty the vector:
Size = 0
Capacity = 4000

Theoretical max number of vector elements = 2305843009213693951

Empty vector:
Size = 0
Capacity = 0

Vector for 1,000 integers:
Size = 1000
Capacity = 1024

Vector for 2,000 integers:
Size = 2000
Capacity = 2048

Vector for 3,000 integers (reserve):
Size = 2000
Capacity = 3048

Shrink vector to fit only current size:
Size = 2000
Capacity = 2000

Add one more element to the vector:
Size = 2001
Capacity = 4000
Size = 0
Capacity = 4000

- What if you always get the same values for `capacity()` and `size()`?

In my first attempt used `void prt(vector<int>);` and `capacity()` was always the same as `size()`. Only when changing from pass-by-value to pass-by-reference was the capacity preserved:
`void prt(vector<int>&)`

- **size vs. capacity in `std::vector`:**
 1. **Size:** Number of elements currently stored (e.g., 2000).
 2. **Capacity:** Total allocated memory for elements (e.g., 2048), always $\geq$ size.
 3. **Reason:** Vectors pre-allocate extra space to optimize performance, avoiding frequent reallocations.
 4. **Growth:** Capacity doubles (e.g., 1024 to 2048) when size exceeds it, using a power-of-2 strategy.
 5. **Control:** Use `reserve()` to pre-allocate space or `shrink_to_fit()` to match capacity to size.

19 PROJECT Modifying containers while iterating [CSC 450]

- Here is a surprisingly hard problem: Removing even numbers from an integer vector. It's easy just printing the non-even elements but it's not easy to remove them if the number of even elements (and hence the size of the resulting array) is not known beforehand. So in C you have to manage memory yourself and pass through the array twice.

Solution that just prints the non-even elements:

```
const int SIZE = 8;
int arr[SIZE] {1,2,2,4,5,6,101,22};
for(int i=0;i<SIZE;i++)
{
    if(arr[i]%2==0)
        continue;
    else
        cout << arr[i] << " ";
}
```

1 5 101

Solution that computes the new logical size:

```

const int SIZE = 8;
int arr[SIZE] {1,2,2,4,5,6,101,22};
int j = 0; // logical size of the new array
for(int i=0; i<SIZE; i++)
{
    if( arr[i]%2 != 0)
        arr[j++] = arr[i];
}
for(int i=0; i<j; i++)
    cout << arr[i] << " ";

```

1 5 101

- **Task:** Write a function that removes all even numbers from a `vector<int>` and returns the modified vector. Show that it works.

- **Solution:**

1. Make sure that the even numbers are recognized. The following program prints a ! every time the iterator points at an even element.

```

#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> v {1,2,2,4,5,6,101,22};

    for (auto it=v.begin(); it!=v.end();it++)
    {
        if (*it % 2 == 0) cout << "!";
        cout << (*it) << " ";
    }
    return 0;
}

```

1 !2 !2 !4 5 !6 101 !22

2. Now I would simply use the `erase(it)` function but that does not work ("Segmentation fault") because if you're modifying the vector while iterating, you break the iterator. `erase(it)` erases the current element and returns a new valid iterator but in the loop, `it++` then passes that element, and ultimately points at a non-existing element (hence the error).

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> v {1,2,2,4,5,6,101,22};

    for (auto it=v.begin(); it!=v.end();it++)
    {
        if (*it % 2 == 0) v.erase(it);
        cout << (*it) << " ";
    }
    return 0;
}
```

0 13 40155953525877 85841540160512 240 211016059126943010357 941635383 82555

3. When an element is erased, the iterator `it` needs to be rebuilt using `erase`. If the element is passed over, this is not necessary.

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> v {1,2,2,4,5,6,101,22};

    for (vector<int>::iterator it = v.begin(); it != v.end();)
    {
        if (*it % 2 == 0)
            it = v.erase(it);
        else
            it++;
    }
}
```

```

    ++it;
    }
    for(auto it=v.begin();it!=v.end();it++)
        cout << (*it) << " ";
    return 0;
}

```

1 5 101

4. Move the operation into a function `removeEven`.

```

#include <iostream>
#include <vector>
using namespace std;

void removeEven(vector<int>& v)
{
    for (vector<int>::iterator it = v.begin(); it != v.end();)
    {
        if (*it % 2 == 0)
            it = v.erase(it);
        else
            ++it;
    }
}

void printVector(const vector<int>& v)
{
    cout << endl;
    for(auto it=v.begin();it!=v.end();it++)
        cout << (*it) << " ";
}

int main()
{
    vector<int> vec {1,2,2,4,5,6,101,22};
    printVector(vec);
    removeEven(vec);
    printVector(vec);
    return 0;
}

```

```

1 2 2 4 5 6 101 22
1 5 101

```

5. Simplify the solution with `erase(first, last)` and `remove_if(first, last, value)`. This reorders a container so that unwanted elements are moved to the end, and then returns an iterator to the new logical end. So the elements are being shuffled before `erase` is applied. The `value` is a **lambda** ("anonymous" inline) function. This is generally faster and safer.

```

#include <iostream> // for cout
#include <vector>    // for vector, erase
#include <algorithm> // for remove_if
using namespace std;

void removeEven(vector<int>& v)
{
    v.erase(remove_if(v.begin(),
                      v.end(),
                      [](int x){ return x % 2 == 0; }),
            v.end());
}

void printVector(const vector<int> v)
{
    cout << endl;
    for(auto it=v.begin(); it!=v.end(); it++)
        cout << (*it) << " ";
}

int main()
{
    vector<int> vec {1,2,2,4,5,6,101,22};
    printVector(vec);
    removeEven(vec);
    printVector(vec);
    return 0;
}

```

```
1 2 2 4 5 6 101 22
1 5 101
```

20 Bonus: What does "theoretical storage limit" mean?

- On my machine, the `max_size()` function returns the number 2305843009213693951 - the theoretical limit for the number of integer elements that my `vector` can hold. How many bytes are this, and how realistic is this limit?

```
cout << fixed << scientific;
cout << static_cast<double>(2305843009213693951 *
                           static_cast<double>(sizeof(int)));
```

```
9.223372e+18
```

- Result:

These numbers are of the order of Exabytes (2^{60}) compared to Gigabytes (2^{30}). There're currently no computers with that size of RAM. Supercomputers have (shared) RAM of the order of Petabytes only.

- How much RAM do you have?

```
awk '/MemTotal/ {printf "%d\n", $2 * 1024}' /proc/meminfo
```

```
67171004416
```

- On Linux systems, a simpler command is `free` (for humans `-h`):

```
free -h
```

	total	used	free	shared	buff/cache	available
Mem:	62Gi	5.7Gi	52Gi	198Mi	4.8Gi	52Gi
Swap:	15Gi	0B	15Gi			

21 PRACTICE Review questions

Add question on header file location, relative and absolute path.

1. Write a statement that defines an empty **vector** object named **avect** that can hold strings. Show that the vector is actually empty.

```
vector<string> avect;  
cout << boolalpha << "The vector is empty: " << avect.empty();  
cout << "\nThe vector is empty: size() = " << avect.size();
```

```
The vector is empty: true  
The vector is empty: size() = 0
```

2. Write a statement that defines a **vector** object named **avect** that can hold integers. The **vector** should have ten elements initialized with the default value 0. Display the elements to show success.

```
vector<int> avect(10,0);  
for (const auto& element : avect) cout << element << " ";
```

```
0 0 0 0 0 0 0 0 0 0
```

3. Write a statement that defines a **vector** object named **avect** that can hold integers. The **vector** should have 100 elements, each initialized with the value 1. If this worked, print a success message, but if any value is not 1, print an error message.

```
vector<int> avect(100,1);  
for (const auto& element : avect)  
    if (element != 1) cout << "Error!" << endl;  
cout << "Initialized to 1!" << endl;
```

```
Initialized to 1!
```

4. Write a statement that defines a **vector** object named **v1** that can hold integers. The vector should be a copy of another vector **v2**. Display both vectors to show that the copying worked.

```
vector<int> v2 {1,2,3};
vector<int> v1(v2);
for (const auto& element: v1) cout << element << " ";
cout << endl;
for (const auto& element: v2) cout << element << " ";
```

```
1 2 3
1 2 3
```

5. What happens when you use an invalid index with the `at` function of the `vector` class? Try it.

```
vector<string> names {"Jim", "Jill", "Jack"};
cout << names.at(0); // valid index
cout << names.at(3); // invalid index
```

An `out_of_range` exception is thrown:

```
terminate called after throwing an instance of 'std::out_of_range'
what(): vector::_M_range_check: __n (which is 3) >= this->size() (which is 3)
```

6. If your program will add a lot of objects to a `vector`, is it best to use the `insert` member function, or the `emplace` member function, and why?

Using the `insert` (and the `push_back`) member function may create copies of objects when inserting them into the `vector`, especially if you pass in fully constructed objects. In contrast, `emplace` (and `emplace_back`) constructs the object in-place, directly in the vector's memory, avoiding unnecessary copying or moving.

This is typically more efficient, especially when dealing with complex objects. Therefore, `emplace` is preferred when adding many objects, particularly if they can be constructed directly with constructor arguments.

7. Internally, how does a `vector` store its elements?

A **vector** stores its elements in a contiguous block of memory on the heap (the "free store") just like a dynamically allocated C-style array. It manages memory automatically and resizes as needed by allocating a larger array and copying existing elements when **capacity** is exceeded.

8. The **vector** class has a **size** and a **capacity** member function. What is the difference between these two member functions?

The **size** function returns the current size of the **vector** (in number of elements) while the **capacity** function returns the number of elements that can be stored before resizing is needed. It can be changed with the **reserve** function.

9. What is the difference between the **vector** class member functions **insert** and **push_back**? Show it with a program.

```
// Define an empty vector of integers.
vector<int> v;
// Add five numbers 1,2,3,4,5 to the vector.
for(int i=0;i<5;i++) v.push_back(i+1);
// Display vector content.
for(const auto& elem : v) cout << elem << " ";
// Create an iterator pointing at the position before 4
auto it = v.begin() + 2;
// Show the iterator value.
cout << "\nIterator positioned before: " << v.at(*it);
// Insert two numbers using the iterator.
v.insert(it, {40,50});
// Display vector content.
cout << endl;
for(const auto& elem : v) cout << elem << " ";
```

```
1 2 3 4 5
Iterator positioned before: 4
1 2 40 50 3 4 5
```

Careful: **push_back** dynamically changes the underlying array. As a result, iterators are invalidated. So this leads to segmentation fault:

```
vector<int> v {0}; // vector has one element v[0]=0
for (auto it = v.begin(); it != v.end(); ++it) // iterator points at end
{
    v.push_back(*it); // modifies the vector while iterating over it
}
```

22 Glossary

TERM	EXPLANATION
Fill constructor	<code>vector<dataType> name(size, value);</code>
Range constructor	<code>vector<dataType> name(iterator1, iterator 2);</code>
Copy constructor	<code>vector<dataType> name(vector2);</code>
Element access	Functions like <code>at(index)</code> or <code>operator[]</code>
Iterator functions	Support functions like <code>begin()</code> and <code>end()</code>
Capacity functions	Size management like <code>size()</code> , <code>empty()</code>
Modifiers	Like <code>push_back(val)</code> , <code>pop_back()</code> , <code>clear()</code>
Comparison operators	Like <code>operator=</code> and <code>operator<=></code>